

Manipulação de erros

O código Dart pode lançar (throw) ou pegar (catch) exceções ou erros.

```
void main() {  
    throw 'Erro!';  
}
```

Throw

Quando acontecer uma exceção:

```
void main() {  
    throw FormatException('Faltou um argumento!');  
}
```

try

Podemos usar `try...on` para executar algo que pode gerar um erro:

```
void main() {  
  try {  
    adicionarTarefa();  
  } on SemTarefa {  
    criarTarefa();  
  }  
}
```

- aula9_throw1.dart

catch

catch pode receber o erro como argumento.

```
void main() {  
  try {  
    adicionarTarefa();  
  } on SemTarefa catch(e) {  
    print('Ouve um erro: $e');  
    criarTarefa();  
  }  
}
```

- aula9_throw2.dart

finally

finally sempre será executado, com ou sem erro.

```
void main() {  
  try {  
    adicionarTarefa();  
  } on SemTarefa catch(e) {  
    print('Ouve um erro: $e');  
    criarTarefa();  
  } finally {  
    terminarTarefas();  
  }  
}
```

- aula9 throw3.dart

Programação assíncrona

O programa continua enquanto aguarda uma operação terminar.

Operações assíncronas

- Buscar dados da internet
- Ler e escrever arquivos
- Acessar banco de dados

Resultado

O resultado de uma operação assíncrona é:

- `Future`
- `Stream` (se tiver várias partes)

A programação assíncrona usa:

- `Future`
- `async`
- `await`

Exemplo imediato

```
String reservaMesa() {  
    var mesa = buscaMesa();  
    return 'Sua mesa é: $mesa';  
}
```

```
String buscaMesa() {  
    return '7';  
}
```

```
void main() {  
    print(reservaMesa());  
}
```

Prática

Altere o método `buscaMesa()` para retornar na String um número aleatório entre 1 e 10.

Solução

```
String buscaMesa() {  
    var aleatorio = Random().nextInt(10) + 1;  
    return '$aleatorio';  
}
```

Exemplo com atraso sem espera

```
import 'dart:math';

String reservaMesa() {
  var mesa = buscaMesa(); // falta esperar pelo futuro!
  return 'Sua mesa é: $mesa';
}

Future<String> buscaMesa() {
  return Future.delayed(const Duration(seconds: 2), () {
    var aleatorio = Random().nextInt(10) + 1;
    return '$aleatorio';
  }));
}

void main() {
  print(reservaMesa());
}
```

Future

Uma Future é o resultado de uma operação assíncrona. Possui dois estados:

- uncompleted
- completed

Usando future

Se usar future uncompleted: Future<T>

Se usar future completed: T ou Error

Exemplo com espera

```
import 'dart:math';

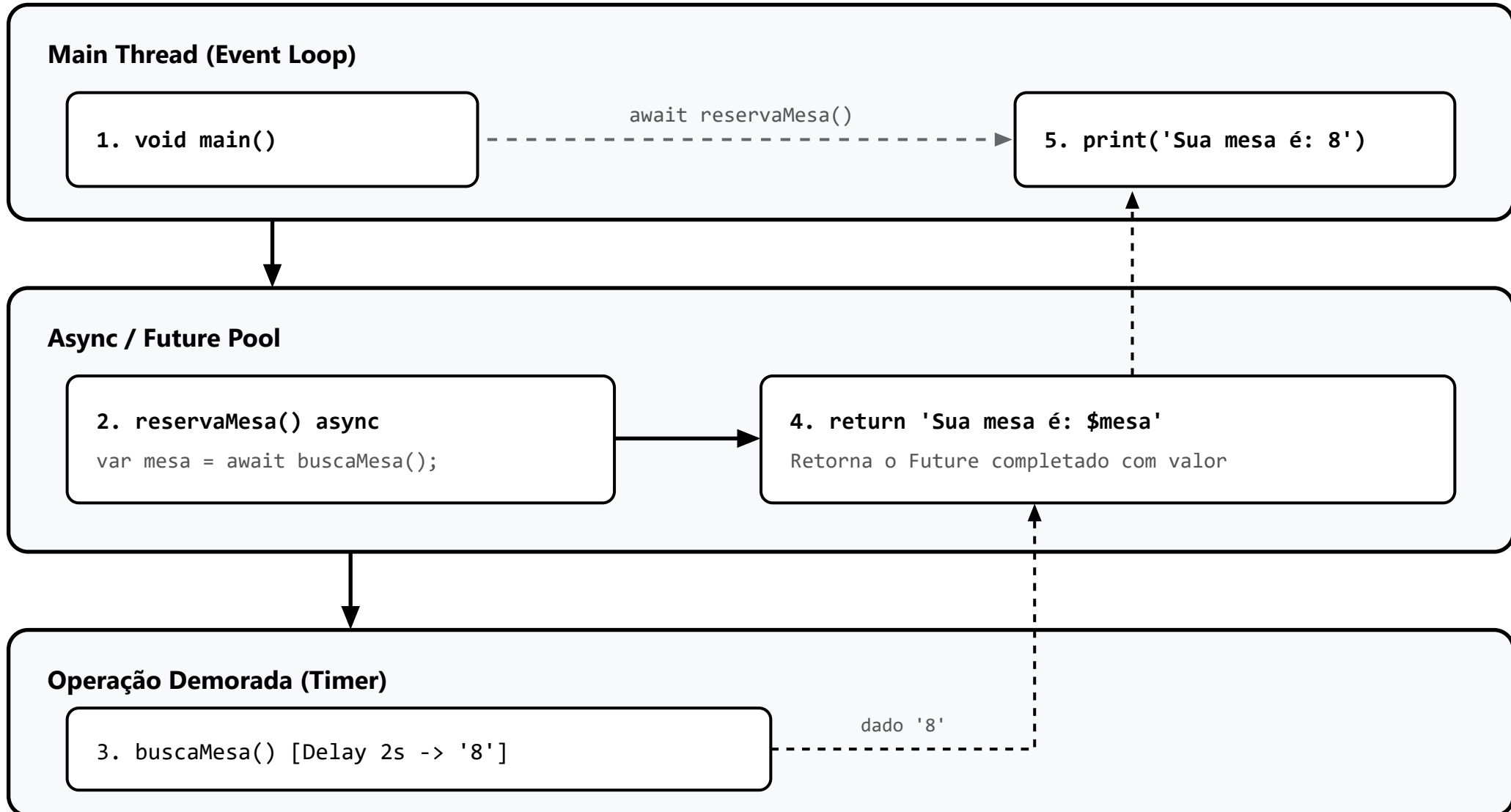
Future<String> reservaMesa() async {
  var mesa = await buscaMesa();
  return 'Sua mesa é: $mesa';
}

Future<String> buscaMesa() {
  return Future.delayed(const Duration(seconds: 2), () {
    var aleatorio = Random().nextInt(10) + 1;
    return '$aleatorio';
  }));
}

void main() async {
  print(await reservaMesa());
}
```


Fluxo Assíncrono: reservaMesa()

`void main() async -> reservaMesa() -> buscaMesa()`



Prática

Altere o método `buscaMesa()` para retornar na String um número aleatório entre 1 e 25.

async e await

A espera `await` só funciona em funções `async`.

```
Future<String> reservaMesa() async {  
    var mesa = await buscaMesa();  
    return 'Sua mesa é: $mesa';  
}
```

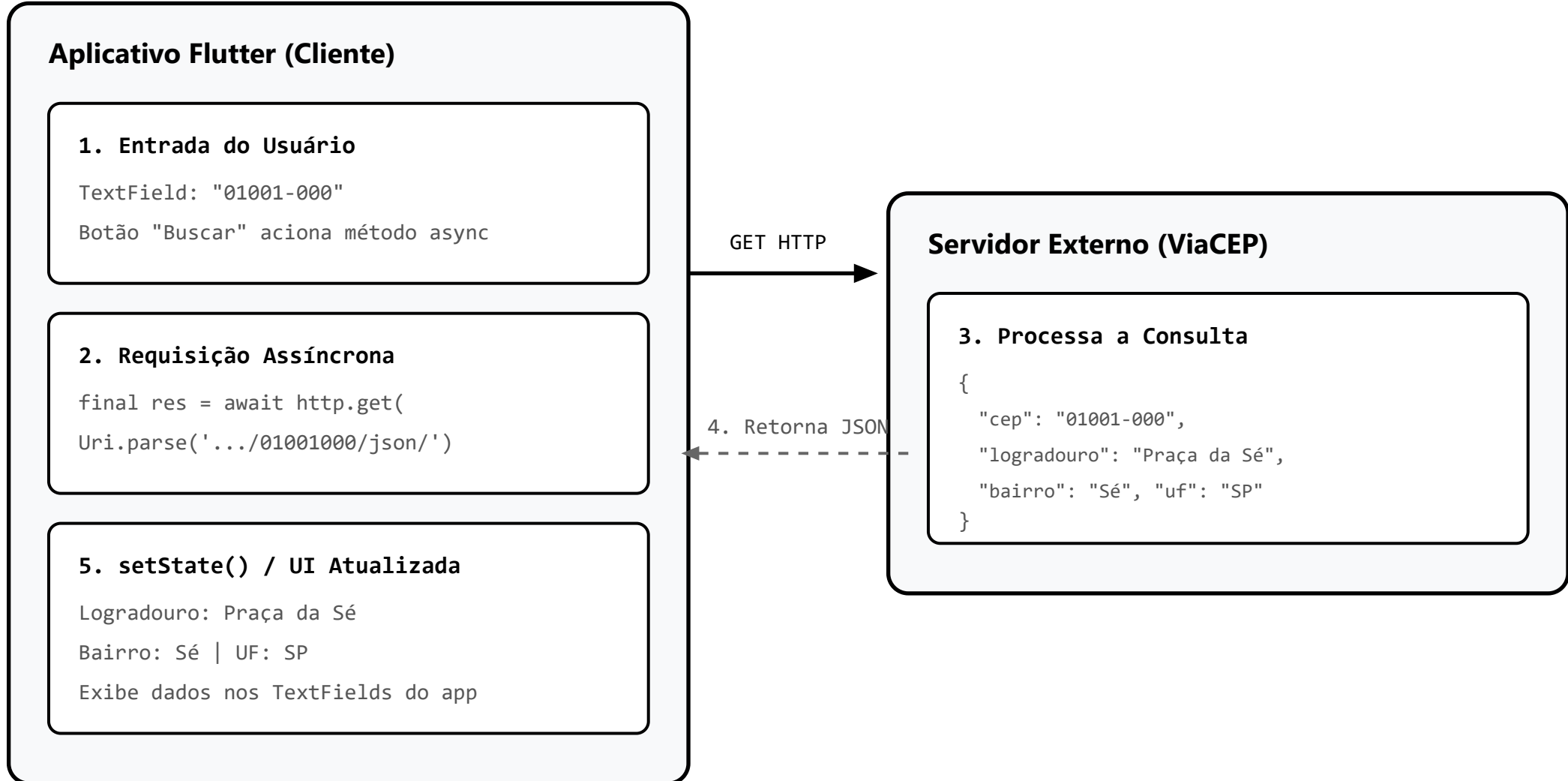
Dados na internet

A app conecta a um servidor e recebe uma resposta:

- <https://jsonplaceholder.typicode.com/guide/>

Integração Flutter + API ViaCEP

TextField (CEP) -> http.get() -> JSON Response -> UI Update



Post

Para receber o JSON:

```
{  
  "userId": 1,  
  "id": 1,  
  "title": "sunt aut ...",  
  "body": "quia et suscipit..."  
}
```

Acesse: <https://jsonplaceholder.typicode.com/posts/1>

Classe compatível com JSON

```
class Postagem {  
  final int userId;  
  final int id;  
  final String title;  
  final String body;  
  
  const Postagem(  
    {required this.userId, required this.id, required this.title,  
    required this.body});  
  
  factory Postagem.fromJson(Map<String, dynamic> json) {  
    return switch(json) {  
      {'userId': int userId, 'id': int id, 'title': String title,  
       'body': String body} => Postagem(  
        userId: userId,  
        id: id,  
        title: title,  
        body: body,  
      ),  
      _ => throw const FormatException('Falha no carregamento...'),  
    };  
  }  
}
```

Buscando dados da internet

Adicione o pacote [http](#)

Execute no terminal:

```
flutter pub add http
```


Buscando dados com `http.get()`

```
Future<http.Response> buscaPostagem() {  
    return http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts/1'));  
}
```

Busca completa

```
Future<Postagem> buscaPostagem() async {  
  final resposta = await http.get(  
    Uri.parse('https://jsonplaceholder.typicode.com/posts/1'),  
    headers: {'Accept': 'application/json'},  
  );  
  
  if (resposta.statusCode == 200) {  
    return Postagem.fromJson(jsonDecode(resposta.body) as Map<String, dynamic>);  
  } else {  
    // status != 200 é erro!  
    throw Exception('Falha ao carregar post.');  }  
}
```

Buscando dado

```
class _MyAppState extends State<MyApp> {  
  
  late Future<Postagem> postFuturo;  
  
  @override  
  void initState() {  
    super.initState();  
    postFuturo = buscaPostagem();  
  }  
  // ...  
  
}
```

Mostrar os dados

```
FutureBuilder<Postagem>(
  future: postFuturo,
  builder: (context, snapshot) {
    if (snapshot.hasData) {
      return Text(snapshot.data!.title);
    } else if (snapshot.hasError) {
      return Text('${snapshot.error}');
    }

    return Text('Carregando...');
  },
)
```

Exemplo completo

- aula9_codigo1.dart

Prática

Ajuste a aplicação anterior para mostrar o próximo post quando o usuário apertar um botão.

Prática

Ajuste a aplicação anterior para mostrar `body` também.